Mini Project: HPC & Computer Vision

- Accelerating IoU computation with Numba
- Sune Kaae
- Track C: JIT Compilation

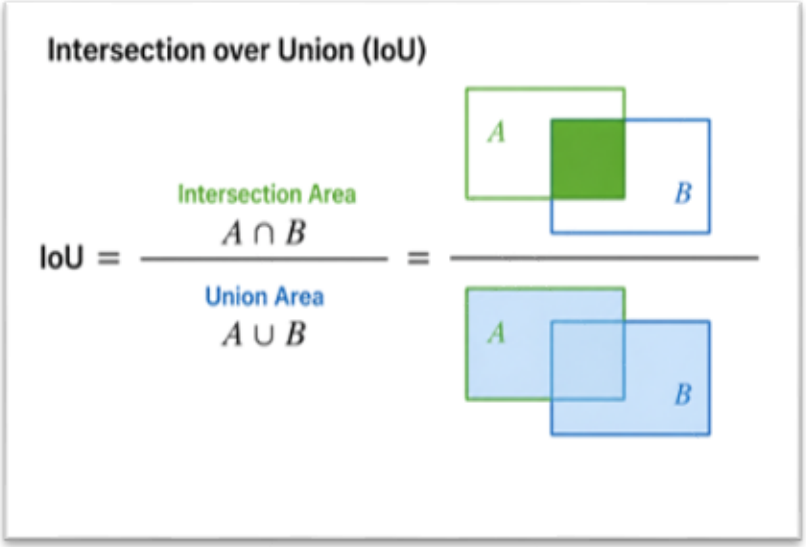
Numba

Motivation

- Work with self-driving trucks
- Camera-based perception
- Bounding boxes for object detection
- IoU used to compare detections

Problem

- Pairwise IoU for all bounding boxes
- Requires all pair comparisons
- Time complexity: O(N^2)
- Implemented with nested Python loops



```python
def compute_iou_python(box_a, box_b):
    # box: numpy [x_min, y_min, x_max, y_max]
    # Image coordinate system; origin (0,0) is top-left. X increases to the right. Y increases downwards
    x_left = max(box_a[0], box_b[0])
    y_top = max(box_a[1], box_b[1])
    x_right = min(box_a[2], box_b[2])
    y_bottom = min(box_a[3], box_b[3])

    inter_w = max(0.0, x_right - x_left)
    inter_h = max(0.0, y_bottom - y_top)
    intersection = inter_w * inter_h

    area_a = max(0.0, box_a[2] - box_a[0]) * max(0.0, box_a[3] - box_a[1])
    area_b = max(0.0, box_b[2] - box_b[0]) * max(0.0, box_b[3] - box_b[1])

    union = area_a + area_b - intersection

    if union <= 0.0:
        return 0.0

    return intersection / union


def iou_matrix_python(boxes):
    n = boxes.shape[0]
    out = np.zeros((n, n), dtype=np.float32)

    for i in range(n):
        for j in range(n):
            out[i, j] = compute_iou_python(boxes[i], boxes[j])

    return out
```

Approach

- Baseline: Python nested loops
- Optimized: Numba JIT (@njit)
- Same algorithm, different execution

```
@njit
def compute_iou_numba(box_a, box_b):
```

```
@njit
def iou_matrix_numba(boxes):
```

```python
from numba import njit

# ---------------------------
# Numba IoU implementation
# ---------------------------
@njit
def compute_iou_numba(box_a, box_b):
    x_left = max(box_a[0], box_b[0])
    y_top = max(box_a[1], box_b[1])
    x_right = min(box_a[2], box_b[2])
    y_bottom = min(box_a[3], box_b[3])

    inter_w = max(0.0, x_right - x_left)
    inter_h = max(0.0, y_bottom - y_top)
    intersection = inter_w * inter_h

    area_a = max(0.0, box_a[2] - box_a[0]) * max(0.0, box_a[3] - box_a[1])
    area_b = max(0.0, box_b[2] - box_b[0]) * max(0.0, box_b[3] - box_b[1])

    union = area_a + area_b - intersection
    if union <= 0.0:
        return 0.0

    return intersection / union


@njit
def iou_matrix_numba(boxes):
    n = boxes.shape[0]
    out = np.zeros((n, n), dtype=np.float32)

    for i in range(n):
        for j in range(n):
            out[i, j] = compute_iou_numba(boxes[i], boxes[j])

    return out
```

Numba JIT

- Compiles Python to machine code
- Removes per-iteration interpreter overhead
- Effective for loops and numerical code
- First run includes compilation cost

Numba

Workload

- 160 images
- ~1300 bounding boxes
- Repeated 50 times
- Baseline runtime ~5 seconds

ultralytics Ultralytics YOLO

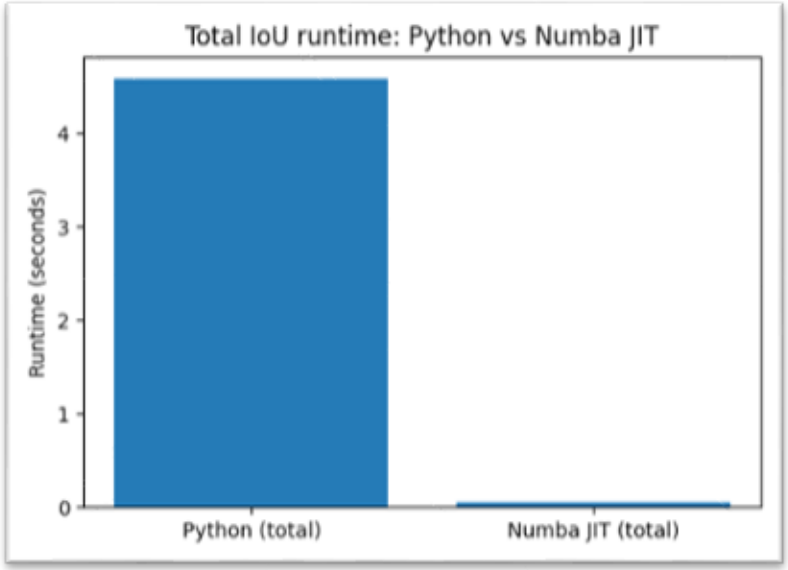
Hardware

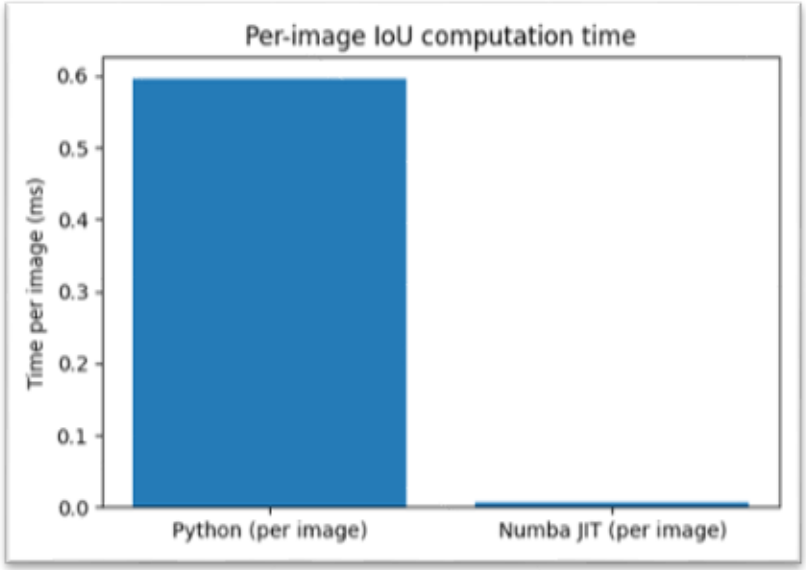
- Google Colab environment
- CPU: Intel Xeon @ 2.20 GHz
- 2 CPUs
- ~1.8 GB RAM

Results (Steady-State)

- Baseline Python: ~4.6 s
- Optimized Numba: ~0.05 s
- Speedup: ~88x

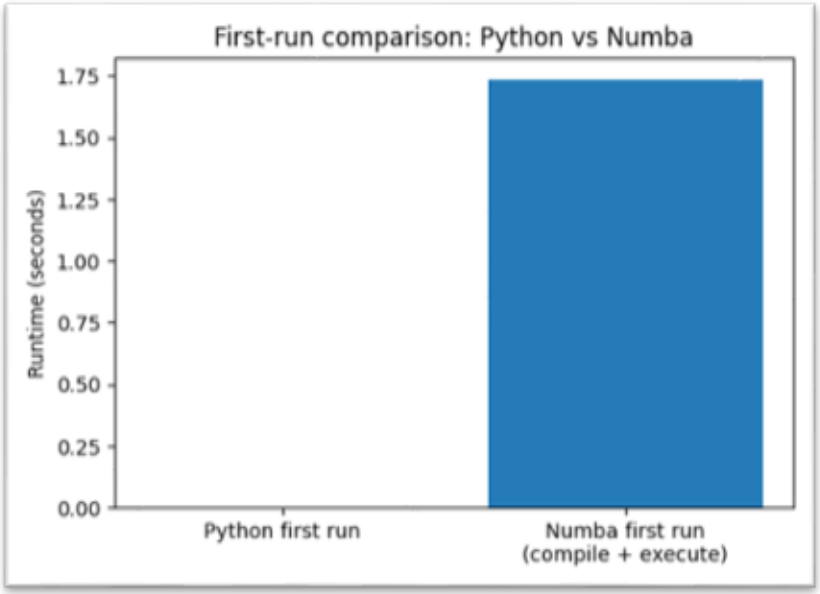
Note: Warm-up run used to exclude compilation time
Note: Same workload and hardware for fair comparison





Results (End-to-End)

- First run: ~1.7 s
- End-to-end speedup: ~2.6x
- Compilation overhead matters

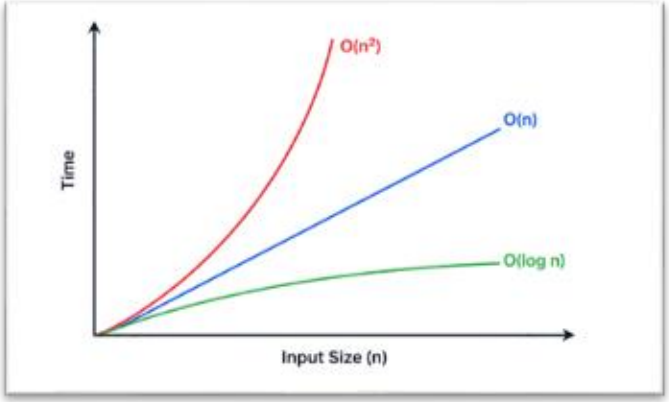


Correctness

- Same results
  - Compared with np.allclose
  - Tolerance: 1e-6
  - Worst-case difference across all outputs was 0.00000024
  - Mean was 0.00000000

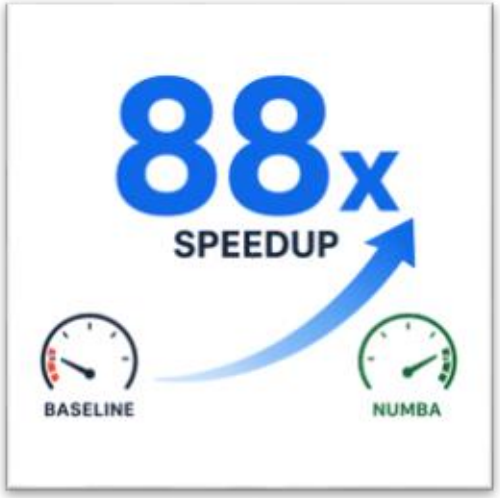
Discussion

- Algorithm remains $O(N^2)$ complexity
- Speedup from reduced constant factor
- Matrix symmetry → ~50% fewer computations possible
- Quick GPU experiment slower than Numba (overhead of many small kernel launches)



Limitations

- Small dataset (160 images), repeated passes
- Focus on IoU kernel, not full pipeline
- CPU focus, only quick exploration of GPU

Conclusion

- Numba gives large speedups (~88x)
- Effective for loop-heavy workloads
- Easy to apply
- Simple optimizations can yield big gains